Best Practices, Folder Structure, Models, Security, Privacy, Performance & Tech Stack

Project Stack

This document defines the recommended technical architecture and engineering standards for the Lebanon Local Commerce & Delivery Platform.

The chosen main stack is:

```
Backend: Spring Boot
Web Dashboards: React
Mobile Apps: React Native
Database: PostgreSQL
Cache / Realtime Support: Redis
Push Notifications: Firebase Cloud Messaging
Maps: Google Maps or Mapbox
Storage: S3-compatible object storage
```

The platform includes:

1. Client Mobile App
2. Driver Mobile App
3. Store / Service Provider Dashboard
4. Admin Dashboard
5. Spring Boot Backend API
6. PostgreSQL Database
7. Redis for cache, driver status, and queues
8. Object storage for images and documents

---

1. Main Architecture Principle

The platform should be built as a **modular monolith first**, not microservices.

Why Modular Monolith First

For the MVP and early versions, a modular monolith is better because:

- It is faster to build.
- Easier to debug.

- Easier to deploy.
- Easier to maintain with a small team.
- Avoids unnecessary microservice complexity.
- Keeps business logic in one place.
- Still allows clean separation by modules.

Future Option

Later, if the platform grows, some modules can be separated into microservices, such as:

- Delivery matching service
- Notification service
- Payment service
- Search service
- Analytics service

But this should happen only when the product has real usage and scaling pressure.

---

2. Recommended High-Level System Architecture

```
Mobile Apps / Web Dashboards
        ↓
API Gateway / Spring Boot Backend
        ↓
Application Modules
        ↓
Domain Services
        ↓
Repositories
        ↓
PostgreSQL Database

Supporting Systems:
- Redis
- Firebase Cloud Messaging
- Object Storage
- Maps Provider
- Email/SMS Provider later
- Monitoring and Logs
```

---

3. Recommended Repository Strategy

Best Choice for Early Development

Use a **monorepo**.

```
lebanon-platform/
  backend/
  apps/
    client-mobile/
    driver-mobile/
    business-dashboard/
    admin-dashboard/
  packages/
    shared-types/
    shared-ui/
    shared-utils/
  docs/
  infra/
```

Why Monorepo

- Easier to manage all apps together.
- Shared API types can be reused.
- Easier to keep frontend and backend aligned.
- Easier for one developer or small team.
- Easier to create consistent UI components.
- Easier to update contracts and models.

When to Split Repositories

Split repositories only when:

- The team becomes larger.
- Different teams own different systems.
- Deployment becomes complex.
- Security boundaries require separation.

---

4. Full Recommended Folder Structure

```
lebanon-platform/
  README.md
  .gitignore
  .editorconfig
  .env.example
  docker-compose.yml

  docs/
    product/
      proposal.md
      functionalities-workflows-api-stack.md
      pages-models-flow-design.md
    architecture/
      best-practices-architecture-stack.md
      api-guidelines.md
      database-design.md
      security-guidelines.md
    decisions/
      ADR-0001-architecture.md
      ADR-0002-auth-strategy.md

  backend/
    pom.xml
    Dockerfile
    src/
      main/
        java/
          com/company/platform/
        resources/
          application.yml
          application-dev.yml
          application-prod.yml
          db/migration/
      test/
        java/
          com/company/platform/

  apps/
    client-mobile/
    driver-mobile/
    business-dashboard/
    admin-dashboard/

  packages/
```

```
    shared-types/
    shared-ui/
    shared-utils/

  infra/
    docker/
    nginx/
    scripts/
    monitoring/
```

---

5. Backend Stack

Main Backend Technologies

```
Java 21+
Spring Boot
Spring Web
Spring Security
Spring Data JPA
PostgreSQL
Flyway
Redis
Bean Validation
MapStruct
Springdoc OpenAPI
JUnit 5
Testcontainers
Docker
```

Recommended Backend Libraries

Core

```
spring-boot-starter-web
spring-boot-starter-security
spring-boot-starter-validation
spring-boot-starter-data-jpa
spring-boot-starter-actuator
spring-boot-starter-websocket
```

Database

```
PostgreSQL Driver
Flyway
Hibernate
```

Mapping

```
MapStruct
```

API Documentation

```
springdoc-openapi
Swagger UI
```

Security / JWT

```
Spring Security
OAuth2 Resource Server
Nimbus JOSE JWT
```

Testing

```
JUnit 5
Mockito
AssertJ
Spring Boot Test
Testcontainers
MockMvc
WireMock
```

Performance / Reliability

```
Redis
Caffeine Cache
Bucket4j or Resilience4j
Micrometer
OpenTelemetry
```

File Storage

```
AWS SDK S3
Cloudflare R2 compatible S3 API
```

Notifications

```
Firebase Admin SDK
```

---

6. Backend Version Strategy

Use stable versions, not random latest versions.

Recommended approach:

```
Java: latest LTS version supported by the chosen Spring Boot version
Spring Boot: latest stable version that is compatible with needed libraries
PostgreSQL: latest stable major supported by hosting provider
Redis: stable managed version
```

Important Rule

Do not upgrade major versions in production without:

- Reading migration guide.
- Checking dependency compatibility.
- Running full automated tests.
- Testing staging environment.
- Backing up database.

---

7. Backend Package Structure

Recommended Spring Boot package structure:

```
backend/src/main/java/com/company/platform/
  PlatformApplication.java

  common/
```

```
    config/
    constants/
    exceptions/
    security/
    utils/
    validation/
    pagination/
    audit/
    mapper/

  modules/
    auth/
    users/
    roles/
    clients/
    stores/
    products/
    inventory/
    orders/
    serviceproviders/
    services/
    servicerequests/
    drivers/
    deliveries/
    payments/
    receipts/
    notifications/
    messaging/
    ratings/
    support/
    admin/
    files/
    reports/
```

Each module should follow a consistent structure.

Example:

```
modules/orders/
  api/
    OrderController.java
    StoreOrderController.java
    ClientOrderController.java
    AdminOrderController.java
  application/
    OrderService.java
```

```
    OrderStatusService.java
    OrderPricingService.java
    OrderTimelineService.java
  domain/
    Order.java
    OrderItem.java
    OrderStatus.java
    PaymentStatus.java
  dto/
    request/
      CreateOrderRequest.java
      RejectOrderRequest.java
    response/
      OrderResponse.java
      OrderDetailsResponse.java
      OrderTimelineResponse.java
  repository/
    OrderRepository.java
    OrderItemRepository.java
  mapper/
    OrderMapper.java
  exception/
    OrderNotFoundException.java
    InvalidOrderStatusException.java
```

---

8. Backend Layering Best Practice

Use clean layers.

```
Controller → Service → Domain Logic → Repository → Database
```

Controller Layer

Responsibilities:

- Receive HTTP requests.
- Validate request body.
- Call service methods.
- Return response DTOs.

Do not put business logic in controllers.

Service Layer

Responsibilities:

- Business rules.
- Status transitions.
- Permission checks.
- Transaction boundaries.
- Calling other modules.

Repository Layer

Responsibilities:

- Database access.
- Queries.
- Persistence.

Do not put business logic in repositories.

DTO Layer

Responsibilities:

- Request validation.
- Response shape.
- Hide internal database fields.

Never expose JPA entities directly to the frontend.

---

9. Backend Naming Conventions

Classes

```
UserController
UserService
UserRepository
UserMapper
UserResponse
CreateUserRequest
UpdateUserRequest
UserNotFoundException
```

Endpoints

Use plural resources:

```
/api/v1/users
/api/v1/stores
/api/v1/orders
/api/v1/deliveries
```

Database Tables

Use snake_case plural names:

```
users
stores
order_items
service_requests
inventory_movements
```

Java Fields

Use camelCase:

```
createdAt
updatedAt
paymentStatus
storeId
```

---

10. Backend Core Models

These are the recommended backend domain models for the platform.

10.1 User Model

```
User
- id
- fullName
- phone
```

```
- email
- passwordHash
- status
- createdAt
- updatedAt
```

Important:

- Phone should be unique if used for login.
- Email can be optional at first.
- Password must always be hashed.
- Never return passwordHash in API responses.

10.2 Role Model

```
Role
- id
- name
- description
```

Roles:

```
ADMIN
SUPPORT_AGENT
CLIENT
STORE_OWNER
STORE_STAFF
STORE_DRIVER
PROVIDER_OWNER
PROVIDER_STAFF
INDEPENDENT_DRIVER
```

10.3 UserRole Model

```
UserRole
- id
- userId
- role
- entityType
```

```
- entityId
- createdAt
```

This allows one user to have multiple roles.

Example:

```
user_id: 10
role: STORE_OWNER
entity_type: STORE
entity_id: 5
```

10.4 Store Model

```
Store
- id
- ownerUserId
- name
- description
- categoryId
- phone
- address
- latitude
- longitude
- deliveryMode
- status
- openingHours
- minimumOrderAmount
- averagePreparationMinutes
- ratingAverage
- ratingCount
- createdAt
- updatedAt
```

Enums:

```
StoreStatus: PENDING_APPROVAL, ACTIVE, SUSPENDED, REJECTED
DeliveryMode: OWN_DRIVER, PLATFORM_DRIVER, BOTH, PICKUP_ONLY
```

10.5 Product Model

```
Product
- id
- storeId
- categoryId
- name
- description
- price
- imageUrl
- isAvailable
- stockStatus
- createdAt
- updatedAt
```

Important:

Product price can change, so orders must store price snapshots.

10.6 Inventory Model

```
Inventory
- id
- storeId
- productId
- quantity
- lowStockThreshold
- reservedQuantity
- updatedAt
```

Important:

`reservedQuantity` can be added later if you want to reserve stock before final completion.

10.7 InventoryMovement Model

```
InventoryMovement
- id
- storeId
- productId
- movementType
```

```
- quantityChange
- previousQuantity
- newQuantity
- reason
- referenceType
- referenceId
- createdByUserId
- createdAt
```

Movement types:

```
STOCK_ADDED
STOCK_REMOVED
ORDER_SALE
ORDER_CANCELLED
MANUAL_ADJUSTMENT
DAMAGED
RETURNED
EXPIRED
```

10.8 Order Model

```
Order
- id
- orderNumber
- clientUserId
- storeId
- status
- subtotal
- deliveryFee
- discount
- total
- paymentMethod
- paymentStatus
- fulfillmentType
- scheduledFor
- notes
- addressSnapshot
- createdAt
- updatedAt
```

Statuses:

```
PENDING
ACCEPTED_BY_STORE
REJECTED_BY_STORE
PREPARING
READY_FOR_PICKUP
DRIVER_ASSIGNED
PICKED_UP
ON_THE_WAY
DELIVERED
COMPLETED
CANCELLED
ISSUE_REPORTED
```

10.9 OrderItem Model

```
OrderItem
- id
- orderId
- productId
- productNameSnapshot
- productImageSnapshot
- quantity
- unitPriceSnapshot
- totalPrice
```

Why snapshots matter:

If the store changes product name or price later, old receipts should stay correct.

10.10 ServiceProvider Model

```
ServiceProvider
- id
- ownerUserId
- name
- description
- categoryId
- phone
- address
- latitude
- longitude
```

```
- status
- openingHours
- responseTimeMinutes
- ratingAverage
- ratingCount
- supportsPickupDelivery
- createdAt
- updatedAt
```

10.11 Service Model

```
Service
- id
- providerId
- categoryId
- name
- description
- pricingType
- basePrice
- isAvailable
- createdAt
- updatedAt
```

Pricing types:

```
FIXED
STARTING_FROM
HOURLY
AFTER_INSPECTION
CUSTOM_QUOTE
```

10.12 ServiceRequest Model

```
ServiceRequest
- id
- requestNumber
- clientUserId
- providerId
- serviceId
- status
```

```
- description
- quoteAmount
- finalAmount
- paymentMethod
- paymentStatus
- deliveryRequired
- preferredTime
- addressSnapshot
- createdAt
- updatedAt
```

10.13 Driver Model

```
Driver
- id
- userId
- driverType
- vehicleType
- phone
- status
- isAvailable
- currentLatitude
- currentLongitude
- ratingAverage
- ratingCount
- completedDeliveriesCount
- createdAt
- updatedAt
```

Driver types:

```
INDEPENDENT
STORE_DRIVER
PROVIDER_DRIVER
```

10.14 Delivery Model

```
Delivery
- id
- deliveryNumber
```

```
- deliveryType
- orderId
- serviceRequestId
- driverId
- status
- pickupName
- pickupAddress
- pickupLatitude
- pickupLongitude
- dropoffName
- dropoffAddress
- dropoffLatitude
- dropoffLongitude
- fee
- distanceKm
- estimatedPickupTime
- estimatedDeliveryTime
- etaConfidence
- paymentCollectionRequired
- cashCollectedAmount
- createdAt
- updatedAt
```

Delivery statuses:

```
WAITING_FOR_DRIVER
DRIVER_ASSIGNED
ARRIVED_AT_PICKUP
PICKED_UP
ON_THE_WAY
ARRIVED_AT_DESTINATION
DELIVERED
FAILED_DELIVERY
CANCELLED
ISSUE_REPORTED
```

10.15 Payment Model

```
Payment
- id
- paymentReference
- paymentType
- orderId
```

```
- serviceRequestId
- deliveryId
- amount
- method
- status
- collectedByDriverId
- collectedAmount
- paidAt
- createdAt
```

Payment methods:

```
CASH_ON_DELIVERY
CARD_LATER
WALLET_LATER
```

Payment statuses:

```
UNPAID
PENDING_COLLECTION
COLLECTED
PAID
DISPUTED
REFUNDED
CANCELLED
```

10.16 Receipt Model

```
Receipt
- id
- receiptNumber
- receiptType
- orderId
- serviceRequestId
- deliveryId
- clientUserId
- storeId
- providerId
- subtotal
- deliveryFee
- discount
- total
```

```
- paymentMethod
- paymentStatus
- generatedAt
```

Receipt types:

```
STORE_ORDER
SERVICE_REQUEST
CUSTOM_DELIVERY
```

10.17 Notification Model

```
Notification
- id
- userId
- title
- body
- type
- referenceType
- referenceId
- readAt
- createdAt
```

10.18 SupportTicket Model

```
SupportTicket
- id
- userId
- relatedOrderId
- relatedDeliveryId
- relatedServiceRequestId
- subject
- description
- status
- priority
- createdAt
- updatedAt
```

10.19 AuditLog Model

```
AuditLog
- id
- actorUserId
- action
- targetType
- targetId
- oldValue
- newValue
- ipAddress
- userAgent
- createdAt
```

Audit logs are important for admin actions, payment changes, inventory changes, and role changes.

---

11. Database Best Practices

Use PostgreSQL

PostgreSQL is the best choice for this platform because the app has many relationships:

- Users to roles
- Stores to products
- Products to inventory
- Orders to order items
- Orders to deliveries
- Payments to receipts
- Service providers to service requests

Use Migrations

Use Flyway for all database schema changes.

Rules:

- Never manually edit production schema.
- Every schema change must be a migration file.
- Migration files must be committed to Git.
- Never edit an old migration after it has been applied to shared environments.

Migration example:

```
V1__create_users_table.sql
V2__create_stores_table.sql
V3__create_products_and_inventory.sql
```

Use UUID or ULID IDs

Recommended:

```
UUID for database IDs
Readable numbers for orders/receipts
```

Example:

```
Database id: 550e8400-e29b-41d4-a716-446655440000
Order number: ORD-000928
Receipt number: R-000394
```

Add Important Indexes

Add indexes for:

```
users.phone
users.email
stores.status
stores.category_id
products.store_id
orders.client_user_id
orders.store_id
orders.status
deliveries.driver_id
deliveries.status
service_requests.provider_id
notifications.user_id
```

Avoid Deleting Important Records

Use soft delete for important business data:

- Stores
- Products
- Users

- Orders
- Receipts
- Payments

Example:

```
deleted_at
is_deleted
```

Never hard-delete financial records.

---

12. API Best Practices

API Versioning

Use:

```
/api/v1
```

REST Naming

Good:

```
GET /api/v1/stores
GET /api/v1/stores/{storeId}/products
POST /api/v1/client/orders
```

Avoid:

```
/getStores
/createNewOrder
/doDeliveryAction
```

Response Format

Use consistent response format:

```
{
  "success": true,
```

```
  "data": {},
  "message": "Success"
}
```

For errors:

```
{
  "success": false,
  "message": "Product is out of stock",
  "code": "PRODUCT_OUT_OF_STOCK",
  "errors": []
}
```

Pagination

Use pagination for lists:

```
GET /api/v1/stores?page=1&size=20
```

Response:

```
{
  "items": [],
  "page": 1,
  "size": 20,
  "totalItems": 150,
  "totalPages": 8
}
```

Filtering

Example:

```
GET /api/v1/admin/orders?
status=PENDING&storeId=123&from=2026-05-01&to=2026-05-02
```

Do Not Trust Frontend Calculations

The backend must calculate:

- Product prices

- Subtotals
- Discounts
- Delivery fee
- Total
- Payment amount
- Inventory availability

The frontend only displays calculations for user experience.

---

13. Backend Security Best Practices

Security is critical because the platform handles users, orders, payments, delivery locations, cash collection, and business data.

13.1 Authentication

Use:

```
JWT access token
Refresh token
Secure password hashing
```

Best practices:

- Access tokens should be short-lived.
- Refresh tokens should be longer-lived but revocable.
- Store refresh tokens securely.
- Rotate refresh tokens after use.
- Hash passwords with BCrypt or Argon2.
- Rate-limit login attempts.
- Add OTP later for phone verification.

13.2 Authorization

Use role-based access control plus entity-level access control.

Role-based check answers:

```
Is this user a STORE_OWNER?
```

Entity-level check answers:

```
Does this STORE_OWNER own this specific store?
```

Both are required.

Example:

```
A store owner can manage products only for stores they own.
```

13.3 Prevent Broken Object Level Authorization

Every endpoint that uses an ID must verify access.

Bad example:

```
GET /api/v1/store-owner/stores/10/orders
```

If the logged-in user owns Store 9, they must not access Store 10.

Correct behavior:

```
Check logged-in user
Check user role
Check store ownership
Then return data
```

13.4 Input Validation

Validate every request.

Use:

```
jakarta.validation
@NotNull
@NotBlank
@Size
```

```
@Email
@Positive
@Min
@Max
```

Example:

```
public class CreateProductRequest {
  @NotBlank
  private String name;

  @NotNull
  @Positive
  private BigDecimal price;
}
```

13.5 Protect Sensitive Data

Never expose:

- Password hash
- Refresh token
- Full internal logs
- Private admin notes
- Driver private documents
- Full payment details
- Unnecessary phone numbers
- Exact location after order is complete

13.6 Secure File Uploads

For product images, documents, or proof uploads:

- Validate file type.
- Limit file size.
- Rename files.
- Store outside the app server.
- Use object storage.
- Scan files later if possible.
- Never trust file extension only.

13.7 Rate Limiting

Rate-limit:

- Login
- Register
- OTP send
- Password reset
- Search
- Public store listing
- Order creation
- Driver location updates

Use Redis-based rate limiting.

13.8 Secure Admin Actions

Admin actions must be logged.

Examples:

- Approving store
- Suspending driver
- Changing payment status
- Refunding payment later
- Editing delivery fee
- Changing user role

13.9 API Security Checklist

Every API should check:

```
Is the user authenticated?
Does the user have the correct role?
Does the user own or have access to this resource?
Is the input valid?
Is the resource in the correct status for this action?
Should this action be logged?
Should this action send a notification?
```

---

14. Privacy Best Practices

The platform will handle private user data such as names, phone numbers, addresses, locations, order history, and delivery behavior.

Collect Only What You Need

Do not collect unnecessary data.

Needed data:

- Name
- Phone
- Delivery address
- Order history
- Payment method
- Driver location during active delivery

Avoid collecting unnecessary personal information.

Location Privacy

Driver and client location must be handled carefully.

Rules:

- Client location is only shared with store/driver for active orders.
- Driver live location is only shown during active delivery.
- Do not show driver location after delivery ends.
- Do not expose exact client location to unrelated stores/providers/drivers.
- Admin access to location should be logged or limited.

Data Retention

Keep business records, but avoid storing unnecessary live tracking forever.

Recommended:

- Keep orders and receipts for business/legal needs.
- Keep payment records.
- Keep audit logs.
- Keep detailed driver location trail for limited time only.
- Keep support tickets as needed.

User Data Control

Later, users should be able to:

- Edit profile.
- Edit addresses.
- Delete saved addresses.
- Request account deactivation.
- Request data deletion where legally possible.

Privacy in Notifications

Push notifications should not reveal too much sensitive information on the lock screen.

Good:

```
Your order is on the way.
```

Avoid:

```
Your expensive medicine order from X pharmacy is on the way to [full address].
```

---

15. Backend Performance Best Practices

Database Performance

- Add indexes early.
- Avoid N+1 queries.
- Use pagination.
- Use projections for list views.
- Do not load full entities when only summary is needed.
- Avoid huge joins in public endpoints.
- Use query optimization for dashboards.

Caching

Use Redis or Caffeine for:

- Categories

- Store summaries
- Home page sections
- Public settings
- Driver availability
- OTP codes
- Rate limits

Do not cache sensitive data carelessly.

Async Jobs

Use async processing for:

- Sending notifications
- Generating receipt PDFs later
- Sending emails later
- Analytics aggregation
- Heavy reports

Realtime Driver Location

Do not write every driver location update directly to PostgreSQL.

Better:

```
Driver app sends location
  ↓
Backend stores latest location in Redis
  ↓
Client tracking reads latest location
  ↓
Important delivery milestones are stored in PostgreSQL
```

---

16. React Web Dashboard Stack

Used for:

- Store dashboard
- Service provider dashboard
- Admin dashboard

Recommended Technologies

```
React
TypeScript
Vite or Next.js
React Router or Next.js routing
TanStack Query
Zustand
React Hook Form
Zod
Tailwind CSS
Shadcn UI
Recharts
Axios or Fetch wrapper
```

Recommended Choice

For dashboards:

```
React + TypeScript + Vite + React Router
```

or:

```
Next.js + TypeScript
```

Simple Recommendation

Use:

```
React + Vite + TypeScript
```

unless you need server-side rendering.

For admin/store dashboards, SSR is usually not necessary.

---

17. React Dashboard Folder Structure

```
business-dashboard/
  src/
    app/
      App.tsx
      router.tsx
      providers.tsx

    assets/

    components/
      ui/
      layout/
      forms/
      tables/
      feedback/

    features/
      auth/
        api/
        components/
        hooks/
        pages/
        schemas/
        types/

      dashboard/
      orders/
      products/
      inventory/
      receipts/
      deliveries/
      services/
      service-requests/
      staff/
      reports/
      settings/

    lib/
      api.ts
      queryClient.ts
      auth.ts
      permissions.ts
      formatters.ts
      constants.ts
```

```
    hooks/
    types/
    styles/
    main.tsx
```

Admin dashboard:

```
admin-dashboard/
  src/
    app/
    components/
    features/
      auth/
      overview/
      users/
      stores/
      providers/
      drivers/
      orders/
      deliveries/
      receipts/
      payments/
      support/
      zones/
      categories/
      reports/
      settings/
    lib/
    hooks/
    types/
```

---

18. React Frontend Best Practices

Use Feature-Based Structure

Group files by feature, not by file type only.

Good:

```
features/orders/
  api/
```

```
  components/
  hooks/
  pages/
  types/
```

Avoid huge global folders like:

```
components/
pages/
services/
```

for everything.

Use TypeScript Strict Mode

Enable strict TypeScript.

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

API Calls

Use TanStack Query for server state.

Use it for:

- Fetching orders
- Fetching products
- Fetching receipts
- Mutations like accept order
- Refetching after status changes

Forms

Use:

```
React Hook Form + Zod
```

Why:

- Clean validation
- Less re-rendering
- Strong types
- Reusable schemas

UI Components

Use:

```
Tailwind CSS + Shadcn UI
```

Build reusable components:

- Button
- Input
- Modal
- Data table
- Status badge
- Empty state
- Page header
- Confirmation dialog
- Timeline
- Receipt card

---

19. React State Management Best Practices

There are two types of state:

Server State

Data from backend:

- Orders
- Products
- Inventory
- Receipts
- Deliveries

Use:

```
TanStack Query
```

Client UI State

Local app state:

- Sidebar open/closed
- Selected filters
- Current role
- Temporary UI flags

Use:

```
Zustand
```

Avoid using Redux unless the app becomes extremely complex.

---

20. React Native Mobile Stack

Used for:

- Client mobile app
- Driver mobile app

Recommended Technologies

```
React Native
TypeScript
React Navigation
TanStack Query
Zustand
React Hook Form
Zod
AsyncStorage
Secure Storage
Firebase Cloud Messaging
Maps library
Location library
Axios or Fetch wrapper
```

Recommended Project Type

For fastest MVP:

```
Expo React Native
```

For more native control:

```
React Native CLI
```

Recommendation

Start with **Expo** unless you already know you need native modules that Expo cannot support.

Expo is useful for:

- Faster setup
- Easier builds
- Easier testing
- Push notifications
- Location features
- Camera/image support

---

21. React Native Folder Structure

```
client-mobile/
  src/
    app/
      App.tsx
      providers.tsx
      navigation/
        RootNavigator.tsx
        AuthNavigator.tsx
        MainTabNavigator.tsx
        OrderNavigator.tsx

    assets/
      images/
      icons/

    components/
```

```
    ui/
    forms/
    cards/
    feedback/
    layout/

  features/
    auth/
      api/
      components/
      hooks/
      screens/
      schemas/
      types/

    home/
    stores/
    products/
    cart/
    checkout/
    orders/
    tracking/
    services/
    custom-delivery/
    receipts/
    favorites/
    profile/
    support/
    notifications/

  lib/
    api.ts
    queryClient.ts
    authStorage.ts
    secureStorage.ts
    permissions.ts
    formatters.ts
    constants.ts

  hooks/
  types/
  theme/
    colors.ts
    spacing.ts
    typography.ts
```

Driver app:

```
driver-mobile/
  src/
    app/
      App.tsx
      providers.tsx
      navigation/

    components/
      ui/
      delivery/
      maps/
      feedback/

    features/
      auth/
      onboarding/
      availability/
      jobs/
      active-delivery/
      cash-collection/
      earnings/
      history/
      profile/
      support/
      notifications/

    lib/
      api.ts
      location.ts
      queryClient.ts
      secureStorage.ts
      permissions.ts
      formatters.ts

    hooks/
    types/
    theme/
```

---

22. React Native Best Practices

Navigation

Use React Navigation.

Recommended navigators:

```
Auth Stack
Main Tabs
Order Stack
Tracking Stack
Profile Stack
```

Driver app:

```
Auth Stack
Driver Main Stack
Active Delivery Stack
Earnings Stack
```

Secure Token Storage

Do not store access tokens in plain AsyncStorage.

Use secure storage:

```
expo-secure-store
react-native-keychain
```

Location Permissions

Ask only when needed.

Client app:

- Ask location when choosing address or nearby stores.

Driver app:

- Ask location when driver goes online.
- Explain why background location may be needed.

Mobile Performance

- Use FlatList for long lists.
- Avoid rendering huge lists at once.
- Memoize expensive components.
- Compress images.

- Use skeleton loading states.
- Avoid unnecessary global state.
- Keep maps screen optimized.

Offline / Bad Network Handling

Lebanon may have unstable connectivity, so mobile apps should handle:

- Slow network
- Failed request retry
- Offline message
- Cached previous data
- Driver action retry for status update

Important driver actions should be retry-safe.

---

23. Shared Frontend Models

Use shared TypeScript types across dashboards and mobile apps.

Recommended package:

```
packages/shared-types/
  src/
    user.ts
    store.ts
    product.ts
    inventory.ts
    order.ts
    delivery.ts
    service-provider.ts
    service-request.ts
    payment.ts
    receipt.ts
    notification.ts
    support.ts
```

Example:

```ts
export type OrderStatus =
  | 'PENDING'
  | 'ACCEPTED_BY_STORE'
  | 'PREPARING'
```

```
  | 'READY_FOR_PICKUP'
  | 'DRIVER_ASSIGNED'
  | 'PICKED_UP'
  | 'ON_THE_WAY'
  | 'DELIVERED'
  | 'COMPLETED'
  | 'CANCELLED';

export interface OrderSummary {
  id: string;
  orderNumber: string;
  status: OrderStatus;
  total: number;
  createdAt: string;
}
```

Important:

Frontend types should match backend response DTOs, not database entities.

---

24. Frontend API Client Best Practices

Create one API client wrapper.

```
lib/api.ts
```

Responsibilities:

- Set base URL.
- Attach access token.
- Handle refresh token.
- Handle errors.
- Normalize response.
- Redirect to login if unauthorized.

Avoid calling `fetch` or `axios` randomly all over the app.

---

25. Frontend Security Best Practices

Do Not Trust Frontend Permissions Alone

Frontend permission checks improve UX, but backend must enforce security.

Example:

Frontend can hide the “Delete Product” button, but backend must still reject unauthorized requests.

Token Storage

Web dashboard:

- Prefer secure HTTP-only cookies if possible.
- If using localStorage, understand XSS risk.
- Use short-lived access tokens.
- Protect against XSS.

Mobile:

- Use secure storage.
- Do not store sensitive info in AsyncStorage.

Hide Sensitive UI Data

Only show phone numbers/contact details when needed for active orders or deliveries.

---

26. Frontend Performance Best Practices

Web Dashboards

- Use code splitting.
- Lazy-load heavy pages.
- Paginate data tables.
- Debounce search inputs.
- Use server-side filtering for large data.
- Avoid loading all orders/products at once.
- Use memoization carefully.
- Optimize images.

Mobile Apps

- Use FlatList.
- Avoid inline heavy functions in list items.
- Cache images.
- Optimize maps.
- Avoid too many live location re-renders.
- Use background tasks carefully.
- Batch status updates where possible.

---

27. Recommended UI Libraries

React Dashboard UI

Recommended:

```
Tailwind CSS
Shadcn UI
Radix UI
Lucide React
Recharts
TanStack Table
```

Use for:

- Admin dashboard
- Store dashboard
- Provider dashboard
- Tables
- Modals
- Forms
- Cards
- Charts

React Native UI

Recommended options:

```
React Native Paper
NativeWind
```

```
Tamagui
Gluestack UI
```

Simple recommendation:

```
NativeWind + custom components
```

Why:

- Similar styling approach to Tailwind.
- Easier to keep mobile and dashboard design language consistent.

---

28. Form and Validation Libraries

Backend

```
Jakarta Bean Validation
Custom validators
```

React Web

```
React Hook Form
Zod
```

React Native

```
React Hook Form
Zod
```

Best practice:

- Frontend validates for UX.
- Backend validates for security and correctness.
- Never rely only on frontend validation.

---

29. Maps and Location Libraries

React Native

If using Expo:

```
expo-location
react-native-maps
```

If using bare React Native:

```
react-native-geolocation-service
react-native-maps
```

Web Dashboards

Options:

```
Google Maps JavaScript API
Mapbox GL JS
Leaflet with OpenStreetMap
```

Backend

Use maps provider APIs for:

- Distance estimation
- ETA estimate
- Geocoding
- Reverse geocoding

Do not overuse paid map APIs. Cache common results where allowed.

---

30. Notifications Best Practices

Use Firebase Cloud Messaging.

Notification types:

```
ORDER_STATUS_UPDATED
NEW_ORDER_FOR_STORE
NEW_DELIVERY_JOB
DRIVER_ASSIGNED
DELIVERY_STATUS_UPDATED
QUOTE_SENT
PAYMENT_COLLECTED
SUPPORT_TICKET_UPDATED
```

Best practices:

- Store notifications in database.
- Send push notification asynchronously.
- Do not block order creation because push failed.
- Keep notification text privacy-safe.
- Allow users to manage notification preferences later.

---

31. Realtime Best Practices

Use WebSockets or Server-Sent Events for:

- Order status updates
- Delivery status updates
- Driver location tracking
- Admin live monitoring

Recommended backend:

```
Spring WebSocket + STOMP
```

Alternative:

```
Socket.IO server as separate Node service later
```

For MVP, you can start with polling for simple order updates, then add realtime.

Recommended MVP Strategy

```
MVP order tracking: polling every 10-15 seconds
MVP driver active delivery: location update every 5-10 seconds
Later: WebSockets for smoother realtime
```

---

32. Testing Best Practices

Backend Testing

Use:

```
JUnit 5
Mockito
Spring Boot Test
Testcontainers
MockMvc
```

Test categories:

- Unit tests for services.
- Integration tests for repositories.
- API tests for controllers.
- Security tests for permissions.
- Transaction tests for inventory updates.

Critical tests:

```
Client cannot access another client's order
Store cannot access another store's inventory
Driver cannot update unassigned delivery
Inventory decreases when order accepted
Inventory restores when order cancelled before pickup
Receipt keeps old price snapshot
Cash collection updates payment status
```

Frontend Testing

Use:

```
Vitest
React Testing Library
Playwright for dashboard E2E
Jest for React Native
React Native Testing Library
Detox later for mobile E2E
```

Critical frontend tests:

- Login flow.
- Order creation flow.
- Store accepts order.
- Driver updates delivery.
- Admin approves store/driver.

---

33. Logging, Monitoring, and Observability

Backend

Use:

```
Spring Boot Actuator
Micrometer
OpenTelemetry
Structured JSON logs
Sentry or similar error tracking
Prometheus/Grafana later
```

Log:

- Request ID
- User ID if available
- Endpoint
- Error code
- Latency
- Important business action

Do not log:

- Passwords
- Tokens
- Full payment info
- Sensitive private messages

Frontend

Use:

```
Sentry
Console logs only in development
```

Track:

- App crashes
- Failed API calls
- Slow screens
- Delivery tracking errors

---

34. Deployment Best Practices

Environments

Use:

```
local
development
staging
production
```

Never test risky changes directly in production.

Backend Deployment

Recommended early setup:

```
Dockerized Spring Boot app
Managed PostgreSQL
Managed Redis
Object storage
```

Frontend Deployment

Dashboards:

```
Vercel
Netlify
Cloudflare Pages
```

Mobile:

```
Expo EAS Build or native build pipeline
```

Secrets

Never commit secrets.

Use environment variables:

```
DATABASE_URL
REDIS_URL
JWT_SECRET
JWT_REFRESH_SECRET
FCM_SERVER_KEY
MAPS_API_KEY
S3_ACCESS_KEY
S3_SECRET_KEY
S3_BUCKET
```

---

35. CI/CD Best Practices

Use GitHub Actions or GitLab CI.

Pipeline should run:

```
Install dependencies
Lint
Type check
Run tests
Build backend
Build dashboards
Run backend integration tests
Build Docker image
```

```
Deploy to staging
Manual approval for production
```

For mobile:

```
Lint
Type check
Unit tests
EAS build or native build
Internal testing release
```

---

36. Code Quality Best Practices

Backend

- Use formatter.
- Use Checkstyle or Spotless.
- Keep services focused.
- Avoid huge classes.
- Avoid circular dependencies.
- Use transactions carefully.
- Use DTOs, not entities, in API responses.
- Keep business rules close to domain services.

Frontend

- Use ESLint.
- Use Prettier.
- Use TypeScript strict mode.
- Avoid huge components.
- Extract reusable UI components.
- Keep feature logic inside feature folders.
- Avoid duplicated API code.

---

37. Status Transition Best Practices

Never allow random status changes.

Example:

A delivery should not move from:

```
WAITING_FOR_DRIVER → DELIVERED
```

Correct path:

```
WAITING_FOR_DRIVER
  ↓
DRIVER_ASSIGNED
  ↓
ARRIVED_AT_PICKUP
  ↓
PICKED_UP
  ↓
ON_THE_WAY
  ↓
ARRIVED_AT_DESTINATION
  ↓
DELIVERED
```

Create status transition validators.

Example service:

```
OrderStatusService
DeliveryStatusService
ServiceRequestStatusService
PaymentStatusService
```

---

38. Inventory Transaction Best Practices

Inventory updates must be safe.

When store accepts order:

```
Start transaction
  ↓
Lock inventory rows for ordered products
  ↓
Check stock quantity
  ↓
Decrease stock
```

```
  ↓
Create inventory movement
  ↓
Update order status
  ↓
Commit transaction
```

If any step fails, rollback everything.

This avoids selling products that are no longer available.

---

39. Payment and Receipt Best Practices

Payment

- Backend controls payment amount.
- Driver can only mark payment collected for assigned delivery.
- Store/admin can see cash collection records.
- Cash mismatch should create issue flag.

Receipt

- Receipt should be generated from backend records.
- Receipt should store snapshots.
- Receipt should not change when product price changes later.
- Receipt number should be unique.

---

40. Admin Best Practices

Admin dashboard must be powerful but safe.

Rules:

- Admin actions should be logged.
- Dangerous actions need confirmation.
- Support agents should have limited permissions.
- Financial changes should require higher permission.
- Admin filters should be strong.
- Admin should see issue alerts early.

Important admin monitoring cards:

```
Delayed orders
Failed deliveries
Cash not settled
Stores with frequent issues
Drivers with repeated failures
Open support tickets
Pending approvals
```

---

41. Suggested MVP Library List

Backend MVP

```
Java 21+
Spring Boot
Spring Web
Spring Security
Spring Data JPA
PostgreSQL Driver
Flyway
Bean Validation
MapStruct
Springdoc OpenAPI
Spring Boot Actuator
Redis
Firebase Admin SDK
JUnit 5
Testcontainers
Docker
```

React Dashboard MVP

```
React
TypeScript
Vite
React Router
TanStack Query
Zustand
React Hook Form
Zod
Tailwind CSS
Shadcn UI
```

```
TanStack Table
Recharts
Lucide React
Axios
```

React Native MVP

```
React Native
Expo
TypeScript
React Navigation
TanStack Query
Zustand
React Hook Form
Zod
Expo Secure Store
Expo Location
React Native Maps
Firebase Cloud Messaging
Axios
NativeWind
```

---

42. What Not to Overbuild in MVP

Avoid these at first:

```
Microservices
Event sourcing
Kafka
Kubernetes
Complex wallet system
Advanced AI recommendations
Full route optimization
Complex fraud engine
Advanced analytics warehouse
Multi-store checkout
Full in-app chat
```

These can be added later.

The MVP should focus on:

```
Authentication
Authorization
Store management
Product and inventory management
Client ordering
Delivery assignment
Driver status updates
Cash collection
Receipts
Admin monitoring
```

---

43. Recommended Development Order

Step 1: Project Setup

- Monorepo setup.
- Backend Spring Boot setup.
- React dashboard setup.
- React Native app setup.
- Shared TypeScript types.
- Docker Compose for local database and Redis.

Step 2: Backend Foundation

- Users.
- Auth.
- Roles.
- Permissions.
- Audit logs.
- API error format.

Step 3: Store + Product + Inventory

- Store model.
- Product model.
- Inventory model.
- Inventory movements.
- Stock status.

Step 4: Client Order Flow

- Store listing.
- Product listing.

- Cart checkout.
- Create order.
- Order details.

Step 5: Store Order Management

- Accept/reject order.
- Preparing/ready.
- Inventory transaction.
- Request delivery.

Step 6: Delivery Flow

- Driver profile.
- Driver availability.
- Available jobs.
- Accept delivery.
- Delivery status updates.
- Cash collection.

Step 7: Receipts and Payments

- Receipt generation.
- Payment status update.
- Receipt details.

Step 8: Admin Dashboard

- Approvals.
- Orders.
- Deliveries.
- Receipts.
- Cash monitoring.

Step 9: Service Provider Flow

- Provider profile.
- Services.
- Service requests.
- Quotes.
- Service receipts.

---

44. Final Engineering Rules

These rules should guide the whole project:

```
Use one shared backend.
Start as a modular monolith.
Use PostgreSQL for business data.
Use Redis for temporary/realtime data.
Use role-based and entity-level authorization.
Never expose database entities directly.
Never trust frontend totals or permissions.
Use transactions for inventory and payment changes.
Use snapshots for order items and receipts.
Log admin and financial actions.
Keep mobile apps fast and simple.
Use feature-based folder structure.
Build MVP first, advanced features later.
```

---

45. Final Recommended Stack Summary

```
Backend:
Java + Spring Boot + Spring Security + Spring Data JPA + PostgreSQL + Redis

Web:
React + TypeScript + Vite + TanStack Query + Zustand + Tailwind + Shadcn UI

Mobile:
React Native + Expo + TypeScript + React Navigation + TanStack Query + Zustand

Database:
PostgreSQL + Flyway migrations

Realtime:
Polling first, then Spring WebSocket / STOMP

Notifications:
Firebase Cloud Messaging

Storage:
S3-compatible storage

Maps:
Google Maps or Mapbox

Testing:
JUnit 5, Testcontainers, React Testing Library, Playwright, React Native Testing
Library
```

```
Monitoring:
Spring Actuator, Micrometer, OpenTelemetry, Sentry
```

This structure gives the project a strong foundation without making the MVP too complicated.